

# Rapport Projet IA03

Détection de personnes et de téléphones en réseau local

**Yves Jahina Chekoua**

**Arnaud Godard**

**Marega Tandjigora**

## Table des matières

|                                                |          |
|------------------------------------------------|----------|
| <b>1 Objectif du projet.....</b>               | <b>2</b> |
| <b>2 Architecture générale.....</b>            | <b>3</b> |
| <b>3 Technologies utilisées.....</b>           | <b>3</b> |
| <b>4 Structure des fichiers.....</b>           | <b>3</b> |
| 4.1 webcam_stream.py (Machine A).....          | 4        |
| L'idée générale :.....                         | 4        |
| Librairies utilisées :.....                    | 5        |
| Point important du code :.....                 | 5        |
| 4.2 person_detection.py (Machine B).....       | 5        |
| Chargement du modèle et connexion au flux..... | 6        |
| Détection des personnes.....                   | 6        |
| <b>5 Mise en réseau.....</b>                   | <b>7</b> |
| <b>6 Difficultés rencontrées.....</b>          | <b>7</b> |
| <b>7 Installation et utilisation.....</b>      | <b>8</b> |
| <b>8 Résultat obtenu.....</b>                  | <b>8</b> |
| <b>9. Fonctionnalités supplémentaires.....</b> | <b>8</b> |

<http://10.174.254.226:5000>

<http://10.174.254.171:5001>

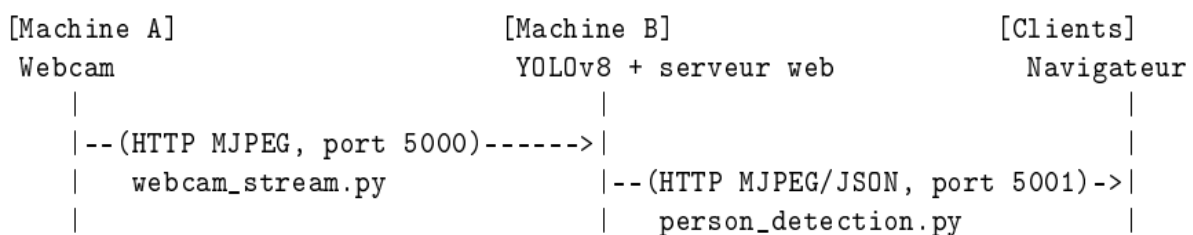
```
python person_detection.py --camera "Entree=http://10.174.254.167:5000/video_feed"  
--camera "Salle=http://10.174.254.226:5000/video_feed" --camera  
"Telephone=http://10.174.254.68:8080/video"
```

# 1 Objectif du projet

Mettre en place un système réparti sur deux machines communiquant sur un réseau local (créé via un point d'accès Wi-Fi partagé depuis un téléphone) :

- une machine capture le ux d'une webcam et le diffuse sur le réseau ;
- une seconde machine récupère ce flux, y applique un modèle d'IA existant (YOLOv8) pour détecter les personnes présentes, et rediffuse à son tour le résultat (vidéo annotée + statistiques) pour qu'il soit consultable depuis n'importe quel appareil du réseau (tablette, téléphone, autre PC).

## 2 Architecture générale



**Machine A** : capture la webcam avec OpenCV, encode chaque image en JPEG, et la diuse en continu via un serveur Flask (ux MJPEG).

**Machine B** : se connecte au ux de la machine A, fait tourner YOLOv8 sur chaque image pour détecter les personnes, dessine les boîtes de détection, puis republie le résultat (image annotée + compteur de personnes + FPS) via son propre serveur Flask, sous forme de ux vidéo et d'API JSON.

**Clients** : (tablette, téléphone, autre PC) : n'importe quel appareil connecté au même réseau peut ouvrir un navigateur et consulter le résultat, sans rien installer. Les deux machines et les appareils clients sont connectés au même point d'accès Wi-Fi, créé en partageant la connexion d'un téléphone (hotspot).

## 3 Technologies utilisées

| Composant                             | Rôle                                             |
|---------------------------------------|--------------------------------------------------|
| Python                                | Langage utilisé pour les deux machines           |
| OpenCV ( <code>opencv-python</code> ) | Capture webcam, lecture de flux, encodage JPEG   |
| Flask                                 | Serveur web léger pour diffuser vidéo et API     |
| Ultralytics YOLOv8                    | Modèle de détection d'objets pré-entraîné (COCO) |
| Threading (Python)                    | Exécuter détection et serveur web en parallèle   |

Le modèle utilisé est **YOLOv8n** (« nano »), la version la plus légère de YOLOv8, pré-entraînée sur le dataset **COCO** (80 classes d'objets courants). On filtre uniquement la classe 0 = *person*.

## 4 Structure des fichiers

machine\_A\_streaming :

webcam\_stream.py -> capture + diffusion du flux webcam (port 5000)  
requirements.txt -> flask, opencv-python 2

machine\_B\_detection :

person\_detection.py -> reception du flux, detection YOLOv8, serveur web (port 5001)  
requirements.txt -> opencv-python, ultralytics, flask

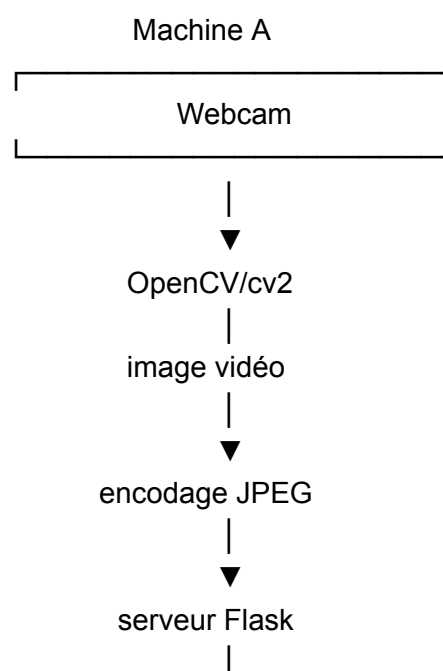
Général projet :

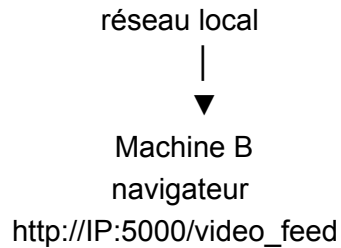
README.md -> guide d'installation et d'utilisation  
documentation.tex -> ce document

### 4.1 webcam\_stream.py (Machine A)

Ce fichier met en place un petit serveur web Flask qui récupère les images d'une webcam avec OpenCV, les transforme en JPEG, puis les envoie en continu sur le réseau local sous forme de flux MJPEG.

L'idée générale :





## Librairies utilisées :

- `python ultralytics` → faire de l'IA / détection d'objets
- `argparse` → permet de passer des paramètres au programme depuis le terminal.
- `socket` → permet de déterminer l'adresse IP locale de la machine.
- `cv2` → OpenCV, utilisé ici pour accéder à la webcam et traiter les images.
- `Flask` → crée le serveur web.
- `Response` → permet à Flask d'envoyer progressivement les images au client.

## Point important du code :

```
def generate_frames(camera_index: int):  
    camera = cv2.VideoCapture(camera_index)  
    if not camera.isOpened():  
        raise RuntimeError(f"Impossible d'ouvrir la camera {camera_index}")  
  
    try:  
        while True:  
            success, frame = camera.read()  
            if not success:  
                break  
  
            ok, buffer = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 60])  
            if not ok:  
                continue  
  
            yield (  
                b"--frame\r\n"  
                b"Content-Type: image/jpeg\r\n\r\n" + buffer.tobytes() + b"\r\n"  
            )  
    finally:  
        camera.release()
```

Utilisation de `yield` au lieu de `return` pour que `generate_frames()`, le générateur produise continuellement des images au lieu de s'arrêter à une seule.

`finally: camera.release()` -> empêche de libérer la caméra.

## 4.2 person\_detection.py (Machine B)

Le fichier `person_detection.py` constitue le programme exécuté sur la machine B. Son rôle est de récupérer le flux vidéo transmis par la machine A et d'effectuer en temps réel une détection des personnes présentes dans les images à l'aide du modèle YOLOv8.

### Chargement du modèle et connexion au flux

Au démarrage, le programme charge le modèle YOLOv8 utilisé pour la détection. Le modèle `yolov8n.pt` est utilisé par défaut. Il s'agit de la version « nano » de YOLOv8, choisie car elle est relativement légère et permet d'obtenir des performances adaptées à une détection en temps réel.

Le programme ouvre ensuite une connexion vers le flux vidéo de la machine A grâce à OpenCV :

```
print("Chargement du modele YOLOv8...")
model = YOLO(model_name)

print(f"Connexion au flux : {stream_url}")
cap = cv2.VideoCapture(stream_url)
```

L'adresse du flux est fournie au programme lors de son lancement. La machine B peut ainsi récupérer directement le flux MJPEG généré par le serveur Flask de la machine A.

### Détection des personnes

Chaque image récupérée depuis le flux est ensuite transmise au modèle YOLOv8 pour analyse. La détection utilise un seuil de confiance de 0,5 par défaut, permettant de limiter les détections aux résultats considérés comme suffisamment fiables. Les résultats sont ensuite utilisés pour générer une image annotée avec les boîtes autour des personnes détectées :

```
results = model(frame, classes=[PERSON_CLASS_ID], conf=conf_threshold, verbose=False)
annotated_frame = results[0].plot()
```

Le programme compte également le nombre de personnes détectées dans chaque image et calcule le FPS (Frames Per Second), c'est-à-dire le nombre d'images traitées par seconde. Ces deux informations sont affichées directement sur la vidéo afin de suivre le fonctionnement du système et ses performances.

Enfin, l'image annotée est affichée dans une fenêtre OpenCV sur la machine B. L'utilisateur peut arrêter le programme en appuyant sur la touche `q`. En cas d'interruption du flux vidéo, le programme tente automatiquement de se reconnecter à la machine A.

Le fonctionnement global de la machine B peut donc être résumé ainsi :

Flux vidéo de la machine A → récupération avec OpenCV → analyse avec YOLOv8 → détection des personnes → annotation de l'image → affichage du nombre de personnes et du FPS.

## 5 Mise en réseau

Le réseau local a été créé via le partage de connexion (hotspot) d'un téléphone. Les deux PC s'y connectent comme à un Wi-Fi classique, puis communiquent entre eux via leurs adresses IP locales (plage 10.174.254.x dans notre cas cette plage dépend du téléphone utilisé).

Étapes suivies :

1. Activation du hotspot sur le téléphone.
2. Connexion des deux PC au même réseau Wi-Fi.
3. Récupération de l'adresse IP de chaque machine (ipconfig sous Windows).
4. Lancement de webcam\_stream.py sur la machine A, puis de person\_detection.py sur la machine B en lui donnant l'URL du flux de A.

## 6 Difficultés rencontrées

| Problèmes                                                                                         | Causes                                                                                                                                             | Solutions                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ping en échec (100% de perte) alors que le réseau fonctionnait / pour le test de la communication | Windows classe les hotspots en profil réseau Public, qui bloque par défaut les requêtes ICMP (ping) entrantes, même quand la machine est joignable | Test direct du vrai ux HTTP (plus fiable que le ping); possibilités alternative : activer manuellement les règles "Demande d'écho ICMPv4 In" dans le pare-feu Windows avancé |
| Confusion entre les IP des deux machines                                                          | le premier ipconfig envoyé était celui de la mauvaise machine                                                                                      | Vérification systématique de quelle machine exécute quelle commande                                                                                                          |
| Popup de pare-feu au premier lancement                                                            | Windows demande confirmation avant d'autoriser une application écouter sur le réseau                                                               | Autoriser Python communiquer sur le réseau lors de la popup                                                                                                                  |
| Manque de fluidité criant sur les vidéos du côté client                                           | Les fichiers vidéos étaient trop importants pour le traitement en temps réel de qualité importante                                                 | Réduction de la qualité de la vidéo distribuée par la machine A de 80 à 60<br>(cv2.imencode( ".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY                                        |

|  |  |             |
|--|--|-------------|
|  |  | LITY, 60] ) |
|--|--|-------------|

## 7 Installation et utilisation

### Machine A

```
pip install -r requirements.txt
python webcam_stream.py
```

### Machine B

```
pip install -r requirements.txt python person_detection.py--stream-url
http://<IP_MACHINE_A>:5000/video_feed
```

Accès depuis un autre appareil (tablette, téléphone, autre PC) sur le même réseau : `http://<IP_MACHINE_B>:5001/`

## 8 Résultat obtenu

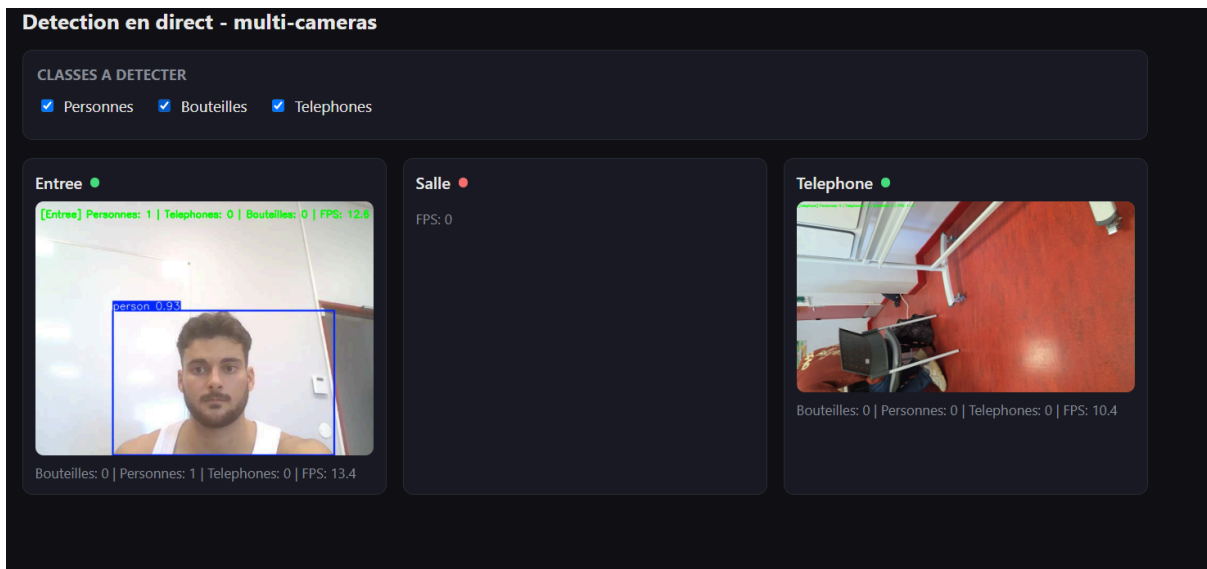
- Diffusion webcam fonctionnelle en temps réel sur le réseau local.
- Détection de personnes fonctionnelles (boîtes de détection et comptage).
- Résultat consultable depuis n'importe quel appareil du réseau via navigateur, sans installation nécessaire côté client.
- API JSON disponible pour de futures extensions (tableau de bord, logs, alertes...).

## 9. Fonctionnalités supplémentaires

Nous avons fait évoluer la solution de plusieurs manière :

1. Nous avons intégré une détection de smartphones et des bouteilles par IA de la même manière que la détection de personne.
2. L'utilisation de plusieurs sources d'images caméra en simultanés qui proviennent de différents appareils (smartphones et PCs) présents sur le réseau local.
3. Une interface intuitive est proposée au client qui lui permet de visualiser l'ensemble des caméras ainsi que de sélectionner "l'objet" que l'IA doit détecter grâce à une checkliste.





Pour l'utilisation de la caméra du smartphone nous avons utilisé l'application "IP Webcam " qui permet de diffuser sur l'adresse IP du mobile la vidéo de la caméra aux appareils présents sur le même réseau local.

Cette interface permet également le décompte du nombre d'occurrence de la classe détecter en temps réel et l'affichage des FPS par caméra.

Ces améliorations permettent d'avoir ainsi une interface qui permet la détection en direct sur plusieurs caméras de plusieurs types d'éléments. Le produit de ce projet s'apparente ainsi à un système de surveillance de vidéos assistés par IA.